

Prompts implementados en “Soporte Bot”

MEJORAR ARQUITECTURA

"Actuá como un Arquitecto de Software y Especialista en Clean Code. Estoy construyendo una aplicación web para el Trabajo Práctico Integrador de la materia Organización Empresarial en la UTN. El sistema emula el proceso de soporte técnico de Nivel 1 mediante un Chatbot basado en una Máquina de Estados Finitos (FSM). El profesor me exige que el diseño no sea un monolito desordenado, sino que respete de manera estricta la separación de responsabilidades. Necesito estructurar el árbol de directorios de modo que la lógica matemática del negocio quede 100% aislada de la manipulación visual del DOM. Además, las credenciales del personal y las soluciones técnicas no deben estar harcodeadas en variables estáticas, sino almacenadas externamente. Proponeme una arquitectura limpia de archivos, explicándome formalmente qué rol cumple cada componente."

Para cumplir con los estándares académicos de ingeniería y garantizar la mantenibilidad del sistema, la solución es implementar un patrón de **Desacoplamiento Estricto (Loose Coupling). La interfaz visual jamás debe conocer las reglas de transición del autómata, y el cerebro del bot jamás debe saber que existe una página web. La arquitectura estructural que debés implementar en el directorio raíz `soporte_bot/` es la siguiente:

Plaintext

```
soporte_bot/
├── index.html      # CAPA DE PRESENTACIÓN: Estructura semántica HTML5 pura (Esqueleto)
├── css/
│   └── style.css   # HOJA DE ESTILOS: Layout elástico y presentación visual media
├── js/
│   ├── bot.js      # LÓGICA PURA (NÚCLEO): Motor de la FSM, switch-case y persistencia
│   └── app.js       # CONTROLADOR DE INTERFAZ: Orquestador del DOM, eventos y peticiones
└── data/
    └── db.json      # MOTOR DE PERSISTENCIA ESTÁTICA: Archivo JSON con datos de configuración
```

Justificación del Rol de Cada Archivo:

1. `data/db.json` (**Datos del Negocio**): Actúa como la base de datos externa de lectura. Almacena las entidades estructuradas `empleados` (legajos autorizados) y el diccionario indexado `soluciones` (árbol de conocimiento). Centralizar la información aquí elimina el código cableado.
2. `js/bot.js` (**Lógica Pura de Caja Negra**): Es el componente más crítico a nivel conceptual. Define el enum `EstadoBot` y la variable de control `estadoActual`. Contiene la función determinística `procesarInput(texto)` que evalúa estados a través de un `switch-case`. **Restricción de diseño:** Este archivo tiene prohibido invocar elementos del DOM (como `document.getElementById`). Solo recibe strings de entrada, transforma el estado de negocio y retorna un objeto de contrato con mensajes y opciones rápidas (`quickReplies`).
3. `js/app.js` (**Orquestador e Interfaz**): Es el puente de comunicación. Se encarga de capturar los eventos del teclado (el clic en enviar o la tecla Enter), limpia las cadenas de texto y se las transfiere al cerebro en `bot.js`. Al recibir el objeto de respuesta, se encarga exclusivamente de pintar las burbujas de chat correspondientes y de actualizar las clases CSS del panel lateral para que la interfaz web refleje fielmente en qué estado se encuentra el autómata. También es el responsable de ejecutar la petición asincrónica `fetch()` al iniciar la aplicación para alimentar la memoria del bot con el archivo JSON.
4. `index.html` e `index.html`: Representan la estructura estática y estética, manteniendo la interfaz desacoplada de los flujos computacionales.**

MEJORA VISUAL

"Actuá como un Diseñador UI/UX experto en CSS estructurado. Tengo una aplicación web para un TPI de la universidad que simula un chatbot de soporte técnico basado en una Máquina de Estados Finitos. Necesito maquetar la interfaz de forma elástica y simétrica utilizando CSS3 puro (Flexbox) sin recurrir a frameworks pesados como Bootstrap. El diseño debe presentar en una fila horizontal el panel de control lateral (`.panel-fsm`) con un ancho fijo de 200px y la ventana del chat (`.chat-y-footer`) ocupando todo el espacio restante de forma flexible. Ambos bloques deben compartir obligatoriamente una altura fija estabilizada de 540px para evitar que la interfaz se desmorone o quede asimétrica al inyectar mensajes dinámicos en el DOM."

"Para lograr un layout elástico, simétrico y de nivel corporativo que resuelva la persistencia visual de la altura, debes implementar un contenedor con dirección de flujo en fila (`flex-direction: row`) y aplicar la propiedad `flex-shrink: 0` en el panel lateral para blindar su ancho estático. El CSS estructurado ideal es:

CSS

```
.contenido-principal {  
  display: flex;  
  flex-direction: row;  
  gap: 16px;  
  width: 100%;  
  height: 540px; /* Estabiliza el layout horizontal */  
}  
.panel-fsm {  
  width: 200px;  
  height: 100%;  
  flex-shrink: 0; /* Evita que el chat comprima el panel */  
  background: #fff;  
  border-radius: 12px;  
}  
.chat-y-footer {  
  display: flex;  
  flex-direction: column;  
  flex: 1; /* Absorbe dinámicamente el espacio restante */  
  height: 100%;  
}
```

Esta configuración garantiza que el cuadro blanco del chat y el panel lateral compartan exactamente la misma línea de base visual, aislando los estilos de la lógica de negocio."

PERSISTIR TICKETS

"Actuá como un Ingeniero de Datos interino. Estoy modelando el flujo 'unhappy path' de un chatbot corporativo en el frontend. Cuando un empleado indica que la solución automatizada no funcionó, el sistema debe derivar el caso creando un ticket de soporte. Sé que debido al Sandbox de seguridad de los navegadores web, no puedo escribir directamente un archivo de texto o modificar un JSON local desde el cliente sin un backend en Node.js o Python. ¿Cómo puedo estructurar código en Vanilla JS para simular fielmente un proceso relacional de inserción de datos (INSERT INTO Tickets_Soporte) en caliente para que sea demostrable y auditable durante una pre-entrega universitaria?"

"Para superar la restricción del Sandbox manteniendo el rigor del modelo de datos de negocio, la solución óptima es implementar una **Persistencia Simulada en Memoria de Sesión Volátil**. Debes declarar una colección indexada global en caliente (`let dbTicketsSimulados = [];`) que actúe como tu tabla relacional. En el estado de excepción, disparará una rutina que parsee y estructure la fila con tipos de datos estándar:

JavaScript

```
function generarTicket() {
  const nroTicket = Math.floor(Math.random() * 9000) + 1000;

  const nuevoRegistro = {
    id_ticket: nroTicket,
    legajo_empleado: empleadoActivo, // Vinculación relacional
    id_falla: opcionSeleccionada,    // FK del incidente
    fecha_creacion: new Date().toLocaleDateString('es-AR'),
    estado: "Abierto - Derivado a Nivel 2"
  };

  dbTicketsSimulados.push(nuevoRegistro); // Simulación de inserción en BD
  console.table(dbTicketsSimulados);     // Renderiza la tabla relacional en consola
  return nroTicket;
}
```

Este enfoque demuestra control metodológico: la información no está suelta en variables aisladas, sino empaquetada bajo un esquema relacional estructurado y auditable en tiempo real mediante las herramientas de desarrollo del navegador."